A Self-Modifying Predictor: Evolving Architecture, Hyper-Parameters, and Structure to a Target Error

Vasili Gavrilov*

2026

Abstract

An ordinary predictor is handed the shape of its network: `predict(Train, Test, Topology, Activation, Jitter, ...)`. The system studied here is handed only the problem and the error it must reach: `self_modifying_predict(Train, Test, Error)`. It discovers the rest by evolution—how many layers and how wide, whether each layer is fully connected or a weight-shared local one, and its own learning rate, momentum, and activation—and it searches until the target error is met. This paper reports what changed from an earlier fixed-budget study and tests one hypothesis: *given a target error, does a self-modifying evolutionary search reach it with less work than random search?* Three things are new. First, each candidate carries its own training hyper-parameters in the genome, and they co-evolve with the topology under a self-adaptive mutation rate. Second, a structural mutation flips a hidden layer between a dense kind and a weight-shared, locally connected *convolution-like* kind, so the search can in principle rediscover on its own the local, shift-invariant inductive bias LeCun built by hand [2]; the convolutional back-propagation is verified against finite differences. Third, the search runs under *error control*—until the best validation error reaches a target—which turns the natural measure from "whose error is lower at a fixed budget" into *time-to-target*: the generations and evaluations each method needs to reach the same error. Every candidate is trained by back-propagation, every run is reproducible from a seed, and every experiment is one shell command. The honest baseline from the earlier study survives: random search is hard to beat, and a self-adaptive mutation rate beats a fixed one by a small, consistent margin. Against that baseline, and over 50 seeds, the self-modifying search reaches a stated target error about as often as random (34 versus 32 of 50) but in fewer generations on the typical run (median 3 versus 4; per-run head-to-head 19 to 11)—a modest edge in the *opposite* direction from the fixed-budget parity, though at this scale it is suggestive (one-sided sign test $p \approx 0.10$) rather than decisive.

1 Introduction

Neural architecture search (NAS) automates a choice a human normally makes: the shape of the network—how many layers, how wide, how connected. Modern NAS methods differ mainly in their *search strategy*: reinforcement learning [6], gradient-based relaxation [9], Bayesian optimization, random search, and evolutionary search [11]. The evolutionary branch is the subject here, in its *evolutionary NAS* form [7, 8]: the topology is evolved while each candidate's weights are trained by gradient descent (as opposed to neuroevolution such as NEAT [4], which evolves weights and topology together without gradients).

Two signatures frame the project. The first is the ordinary one, in which the architecture and training knobs are arguments the user must supply:

```
predict(Train, Test, Topology, Activation, Jitter, ...).
```

*Independent researcher. Physics and computer science by education; a professional software engineer with a background in numerical optimization and machine learning, including rule engines and expert systems. Reference implementation: `https://github.com/Anode1/SMBPANN`.

The second is the one this system aims at, in which the user supplies only the data and the error they are willing to accept:

```
self_modifying_predict(Train, Test, Error).
```

Everything the first signature takes as an argument, the second must discover. An earlier version of this study [12] took the first step—evolving layer counts and widths—but still compared methods the ordinary way: fix a compute budget, run, and see whose final error is lower. This paper closes the gap to the second signature in three moves (co-evolved hyper-parameters, an evolvable convolution-like layer, and error control) and, with error control in place, asks the question the second signature actually poses to a user: not "whose error is lower" but "who reaches *my* error with less work."

A recurring, humbling result in this field is that *random search is hard to beat.* Bergstra and Bengio [5] showed plain random search is a strong baseline for hyper-parameter optimization; Li and Talwalkar [10] showed random search (with weight sharing) is competitive with leading NAS methods, and documented a reproducibility problem in the field along the way. Our contribution is not a new method. It is a small, honest, fully reproducible test of a sharp question, in a system simple enough that every number here can be regenerated from source with one command. The self-adaptive machinery we lean on is standard in the optimization literature the author has worked in—the evolution strategies of Rechenberg and Schwefel [3], in which the step size is itself under selection.

2 Hypothesis

We state the claim to be tested, and its null, plainly.

> **H.** Given only a training set, a test set, and a target validation error, a self-modifying evolutionary search—one that co-evolves architecture and training hyper-parameters and can grow structural (convolution-like) layers—reaches the target using *fewer* candidate evaluations than a matched random search that draws the same kind of random architectures.
>
> **H_0.** It reaches the target using no fewer evaluations than random search (the two are equivalent, or random is at least as fast).

The measure is *time-to-target*: the number of generations (equivalently, evaluations, since each generation is a fixed batch of them) a method needs before its best validation error first drops to the target. A method that never reaches the target within a generous cap is recorded as a non-arrival. This is the quantity a user of `self_modifying_predict` cares about, and it is the quantity the earlier fixed-budget protocol could not measure.

3 Method

3.1 Network and training

A network is a feed-forward stack of dense and convolution-like layers. For a dense layer ℓ with weight matrix $W^{(\ell)}$ and bias $b^{(\ell)}$, the forward pass is

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \qquad a^{(\ell)} = g\Big(z^{(\ell)}\Big), \tag{1}$$

with g a per-network hidden activation—logistic sigmoid $g(z) = 1/(1+e^{-z})$, tanh, or ReLU—chosen by the genome; the output layer is always a dense sigmoid. Weights are trained by back-propagation (the generalized delta rule) with momentum: with local error $\beta_i = (d_i - a_i)\, g'(z_i)$

at output units and $\beta_i = g'(z_i)\sum_k W_{ki}^{(\ell+1)}\beta_k^{(\ell+1)}$ at hidden units, the update is

$$\Delta W^{(\ell)}(t) = r\,\beta^{(\ell)}\big(a^{(\ell-1)}\big)^{\top} + \alpha\,\Delta W^{(\ell)}(t-1), \tag{2}$$

with learning rate r and momentum α, both carried in the genome. Weights are initialized uniformly in $[-2.4/f_i,\ 2.4/f_i]$, where f_i is the fan-in. Training is online (one example at a time), so a worker's memory footprint is bounded regardless of dataset size. All arithmetic uses 32-bit floats; all randomness flows through a seeded 32-bit xorshift generator, so the same seed gives the same result on any machine.

3.2 A convolution-like layer as an evolvable structure

A hidden layer may instead be *convolutional* in one dimension: F filters, each a shared kernel of width K, swept across the previous layer's $d_{\ell-1}$ units. A filter f at position p computes

$$z_{f,p}^{(\ell)} = b_f + \sum_{k=0}^{K-1} w_{f,k}\,a_{p+k}^{(\ell-1)}, \qquad p = 0,\ldots,d_{\ell-1}-K, \tag{3}$$

so the layer's width is *derived*, $d_\ell = F\,(d_{\ell-1}-K+1)$, not a free parameter, and its weight count is only $F\,K$. This is the pair of properties LeCun introduced for images—local receptive fields and weight sharing [2]—here made a *move* the search can take rather than a design the user imposes. Back-propagation through such a layer accumulates each shared weight's gradient over all positions it was applied at, and propagates error backward by the transpose (a full convolution). We verified this against a central finite-difference gradient check: the analytic and numerical gradients agree to within $5\times10^{-3} + 0.02\,|g|$ on every weight, so the structural move trains correctly.

3.3 Genome, co-evolution, and self-adaptation

A *genome* carries everything the second signature must discover:

- the topology: for each hidden layer, a kind (dense or convolutional) and its free parameters (a dense width, or a filter count F and kernel width K);
- the training hyper-parameters: learning rate r, momentum α, and hidden activation;
- a self-adaptive mutation rate σ.

The input and output widths are fixed by the problem. One *mutation* makes a single small change: perturb a hidden layer's free parameter by one; flip a hidden layer between dense and convolutional; or insert or delete one hidden layer (of a random kind), within bounds on depth ($L_{\max}$) and width ($W_{\max}$). After any structural change, derived convolutional widths are recomputed and any kernel that no longer fits its input is clamped, so a genome is always buildable. There is no crossover: recombining two arbitrary topologies is ill-posed (the competing-conventions problem), and mutation-only evolution is the current default in evolutionary NAS [8].

Reproduction is *self-adaptive* in the sense of evolution strategies [3]. An offspring inherits its parent's mutation rate, perturbs it log-normally, and applies that many topology mutations:

$$\sigma' = \sigma\cdot\exp\big(\tau\,\mathcal{N}(0,1)\big), \qquad \text{number of mutations} = \mathrm{round}(\sigma'), \tag{4}$$

clamped to $[\sigma_{\min},\sigma_{\max}]$, with $\tau = 0.35$. The training hyper-parameters co-evolve the same way, by gentler log-normal steps on r and α (each clamped to a sane range) and an occasional switch of activation. Selection then favors not just good topologies but the rates and hyper-parameters that produced them: the rate tends to rise early (to explore) and fall later (to refine), and the learning rate and momentum drift toward values that train the current shape well.

Algorithm 1 Self-modifying search under error control, with matched random control

population $\leftarrow$ P random genomes; both bests $\leftarrow \infty$
for generation $= 1$ to G (safety cap) **do**
  evaluate the population in parallel (one worker process per genome)
  keep the best k (elite); update the genetic best-so-far
  refill: each new genome reproduces self-adaptively from a random elite (topology, hyperparameters, and rate all co-evolve)
  control: evaluate P fresh random genomes; update control best-so-far
  record the first generation each best reaches the target error
  stop once both bests are at or below the target
end for
return generations-to-target for the genetic algorithm and for the control

3.4 Evolutionary search, error control, and the random control

The search keeps a population of P genomes, retaining the best k (elitism) and refilling the rest by self-adaptive reproduction from the elite (Algorithm 1). The *matched random-search control* draws P fresh random genomes every generation and keeps its own best-so-far, so per generation it makes exactly the same number of evaluations as the genetic algorithm. The genetic algorithm's *initial* population is drawn from the same random distribution as the control, so both start from the same place and differ only in what they do next. Every candidate is evaluated by its own worker process; the processes run in parallel, one per CPU, and hand back plain text.

Under *error control*, both methods run until their best validation error first reaches the target, with a generation cap G only as a safety stop. Because the fitness a candidate is scored by *is* its validation error, the target is a single number that plays both roles: the objective each candidate minimizes and the condition that halts the search. For each method the run records the first generation its best crossed the target (or that it never did).

3.5 The task

To ask whether architecture matters, the task must be one where it does—and which the networks can actually learn, so that better architectures score better. We generate a synthetic classification task, reproducibly, from a seed. Inputs x are uniform in $[-1, 1]^d$; the label is

$$y = \big[\, \sin(f\, w \cdot x) > 0 \,\big], \tag{5}$$

for a fixed random projection w and frequency f, with a fraction of labels flipped as noise. Higher f makes more decision stripes and demands more capacity. The projection $w \cdot x$ is a spatial mixing of adjacent inputs, so a local, weight-sharing layer has structure to exploit—which is why the convolution-like move is not idle here. We screened for learnability so that trained networks reach a test error well under the 0.25 a constant 0.5 predictor scores, leaving room for architecture to drive generalization.

3.6 Protocol and metrics

Each *run* uses a task and a search seed; within a run the genetic algorithm and the control see the same task and the same 80/20 train/test split. Two designs are available: a *fresh task per run* (task seed = run index), which samples over task instances as the earlier study did, and a *fixed task* with only the search seed varying, which isolates search efficiency from task-difficulty luck. We report, over the runs and for each method: how often it reached the target (arrivals out of R), and among arrivals the median and mean generations-to-target (equivalently, evaluations, $=$ generations $\times P$). We also report the head-to-head: in how many runs each method reached the target in fewer generations than the other.

Table 1: Self-adaptive versus fixed mutation, small space, 10 seeds. Entries are genetic wins out of 10 (mean test-MSE gap over random). Self-adaptation wins more often and by a wider margin at every starting rate.

starting rate $(-M)$	fixed	self-adaptive
1	5 (+0.0021)	8 (+0.0026)
3	5 (+0.0004)	6 (+0.0017)
6	5 (+0.0009)	7 (+0.0023)

Table 2: Time-to-target over $R = 50$ seeds (fresh task per run), target error 0.17, safety cap 30 generations, $P = 10$ (so evaluations = 10× generations). Medians and means are over the runs that reached the target. The self-modifying search reaches the target about as often as random but in fewer generations on the typical run.

method	reached target	median gens	median evals	mean gens
self-modifying GA	34 / 50	**3**	**30**	6.0
random search	32 / 50	4	40	6.6

4 Experiments

4.1 Correctness of the structural move

Before any search, the convolution-like layer is checked in the unit suite: a forward pass with known shared weights reproduces the hand-computed sums (confirming weight sharing), and the analytic gradient of every weight matches a central finite-difference estimate to within $5 \times 10^{-3} + 0.02\,|g|$. A small convolutional network's training error falls monotonically on a fixed batch. Only then is the move handed to evolution.

4.2 Self-adaptive versus fixed mutation (controlled)

We first isolate the one clean A/B from the earlier study: a fixed mutation rate against the same rate left free to self-adapt, in a small search space ($L_{\max} = 3$, $W_{\max} = 16$), holding everything else equal ($d = 4$, 600 samples, $f = 3$; $P = 10$, $G = 8$; 1200 training epochs; 10 seeds). Only the adaptation is toggled.

Table 1 shows the effect: a fixed rate is about a coin flip against random (5 of 10) whatever the rate, and self-adaptation lifts this to 6–8 of 10 and widens the gap. The benefit is real but modest, and it is why every search below uses the self-adaptive rate.

4.3 Time-to-target: the main test

The central experiment runs the self-modifying search under error control against its matched random control and measures time-to-target. The full search space is enabled (dense and convolution-like layers, co-evolved hyper-parameters), on the task at $d = 8$, $f = 3$, so that a local weight-sharing layer has structure to find. Settings: $P = 10$, elitism $k = 2$, $L_{\max} = 3$, $W_{\max} = 16$, initial rate 2; 1000 training epochs; 300 samples; target error 0.17; a safety cap of $G = 30$ generations; a fresh task per run over $R = 50$ seeds. The target sits, by design, between the error a directed search can reach on this task and the error broad random sampling typically stalls at, so that time-to-target can separate the methods rather than saturate.

Table 2 reports the race. Both methods reach the target on most seeds and about equally often (34 and 32 of 50); the difference is in speed. The self-modifying search reaches the target in fewer generations on the typical seed (median 3 versus 4, mean 6.0 versus 6.6 generations,

i.e. 30 versus 40 evaluations), and it wins the per-run head-to-head 19 to 11, with 8 ties and 12 seeds on which neither method reached the target within the cap. This is a modest edge, and at 50 seeds it is *suggestive rather than decisive*: a sign test on the 30 decisive runs gives a one-sided $p \approx 0.10$. Its direction, however, is the notable part. Under the earlier fixed-budget measure the same kind of search sat at parity or below random; under time-to-target it comes out slightly ahead. The two measures are not in conflict. Fixed-budget final error rewards broad late-stage coverage, which favors random; time-to-target rewards fast early refinement toward a stated error, which favors a directed search that commits to its best few samples and polishes them. The interface `self_modifying_predict` poses the second question, not the first, and by that question the self-modifying search is at least as good as random and somewhat faster.

4.4 Fixed-budget parity, for context

For continuity with the earlier study we note its fixed-budget finding, which the new machinery does not overturn: when the search space is enlarged and the budget held fixed, the genetic algorithm hovers near parity with random search and does not reliably beat it, consistent with [5, 10]. Error control does not change that landscape; it changes the *question*, from final error at a fixed budget to work-to-a-fixed-error, which is the one the target-driven interface poses.

5 Discussion

The structural move is now available to evolution, and it trains correctly: the search can, in principle, assemble local weight-sharing layers rather than being told to. Whether it *should*, on a given task, is exactly what time-to-target measures—and measures more faithfully than fixed-budget final error, because a user of `self_modifying_predict` specifies an error and pays for the work to reach it. Two forces set that work. Directed local search (the genetic algorithm) spends its global sampling budget only in the first generation and refines thereafter, while random search samples globally throughout; so on a small or structured space, or when the target lies where local refinement can climb, the directed search should arrive sooner, and on a large flat space random's broader coverage should. The self-adaptive rate and the co-evolved hyper-parameters are there to help the directed search on the first count without hand-tuning.

Limitations. The scale is small and the task is synthetic; convolution is one-dimensional; the fitness landscape over architectures is fairly flat, which is unfavorable to any directed search; mean squared error stands in for classification error; and time-to-target with a fixed absolute target inherits whatever variation the fresh-task design adds to the achievable floor (the fixed-task design removes it, at the cost of generality). None of this is hidden, and each bounds how far to read the result.

6 Conclusion and future work

The twenty-five-year-old idea—hand the network only the problem and the error, and let it find its own shape—now runs end to end: architecture, training hyper-parameters, and even the choice between a dense and a weight-sharing layer are discovered by evolution, and the search halts when the target error is met. On the hypothesis of Section 2, the evidence leans toward H and away from H_0: the self-modifying search reached the target in fewer evaluations than random on the typical seed and won the head-to-head 19 to 11; but the margin is not significant at 50 seeds ($p \approx 0.10$), so we report a lean, not a rejection. Settling it is the first of three clean next steps. First, the fixed-task time-to-target design at many search seeds, to isolate search efficiency from task luck. Second, a two-dimensional convolution and an image-shaped task, the setting where the local weight-sharing move should pay decisively rather than marginally.

Third, a bridge to a standard tabular NAS benchmark (NAS-Bench-101/201), which would let the same time-to-target comparison run over thousands of seeds cheaply. Each is one command away in the released code.

Reproducibility

Every result above is regenerated from [12] with a single command. The engine builds with `make` (a C99 compiler, no dependencies) into three programs: `smbpann` (the worker), `evolve` (the search and control), and `gentask` (the task generator). The convolutional gradient check and the other correctness tests run as `make ut` under AddressSanitizer and UBSan. `scripts/errortest.sh` runs the time-to-target experiment; its controlling variables are environment variables: `DIM`, `N`, `FREQ`, `NOISE` (the task); `TARGET` (the target error); `LMAX`, `WMAX` (the search space); `POP`, `GENS`, `ELITE`, `MUT`, `ADAPT` (the search); `EPOCHS` and `RUNS`; and an optional `FIXTASK` to hold the task fixed. For example, Table 2 is

```
DIM=8 N=300 FREQ=3 NOISE=2 TARGET=0.17 POP=10 GENS=30 EPOCHS=1000
LMAX=3 WMAX=16 MUT=2 ADAPT=1 ELITE=2 RUNS=50 scripts/errortest.sh
```

and `FIXTASK=1` switches to the fixed-task design. Each run uses a fixed 32-bit xorshift generator, so the numbers are identical on re-running.

References

[1] D. E. Rumelhart, G. E. Hinton, R. J. Williams. Learning representations by back-propagating errors. *Nature* 323, 1986.

[2] Y. LeCun, B. Boser, J. S. Denker, et al. Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation* 1(4), 1989.

[3] I. Rechenberg. *Evolutionsstrategie*. 1973; H.-P. Schwefel, *Numerical Optimization of Computer Models*, 1981 (self-adaptive mutation in evolution strategies).

[4] K. O. Stanley, R. Miikkulainen. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10(2), 2002.

[5] J. Bergstra, Y. Bengio. Random Search for Hyper-Parameter Optimization. *JMLR* 13, 2012.

[6] B. Zoph, Q. V. Le. Neural Architecture Search with Reinforcement Learning. *ICLR* 2017.

[7] E. Real et al. Large-Scale Evolution of Image Classifiers. *ICML* 2017.

[8] E. Real, A. Aggarwal, Y. Huang, Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI* 2019.

[9] H. Liu, K. Simonyan, Y. Yang. DARTS: Differentiable Architecture Search. *ICLR* 2019.

[10] L. Li, A. Talwalkar. Random Search and Reproducibility for Neural Architecture Search. *UAI* 2019.

[11] T. Elsken, J. H. Metzen, F. Hutter. Neural Architecture Search: A Survey. *JMLR* 20, 2019.

[12] V. Gavrilov. SMBPANN: a self-modifying backpropagation feed-forward neural network. Source code, `https://github.com/Anode1/SMBPANN`.